Technical University of Cluj-Napoca

Faculty of Electronics, Telecommunications and Information Technology

Architecture of the Microprocessors

Sensors System Control using Arduino boards

Electronic Level Using MPU-6050 and Arduino

Student:

Bălășcău Patricia-Andreea

Coordinator:

Assist. Prof. Mircea Giurgiu, PhD

Ing. Ioana Maeva Cărbunescu

2025-2026

Contents

1. Introduction 3
1.1. Project Description and Requirements 3
1.2. Block diagram 3
1.2.1. Project Requirements (Tilt Angle Measurement) 3
1.2.2. Proposed Solution: MPU-6050 IMU over I²C 4
1.2.3. Added Features: I²C LCD Display, LEDs, and Buzzer Alarm 4
2. Theoretical Background 5
2.1. MPU-6050 Overview 5
2.2. Accelerometer Principle (Gravity-Based Tilt) 5
2.3. Gyroscope Principle (Angular Rate) 5
2.4. Basic Angle Estimation and Sensor Fusion (Short Overview) 5
3. Implementation 6
3.1. Hardware Implementation 6
3.1.1. Arduino Board 6
3.1.2. MPU-6050 Connections (Power and I²C) 6
3.1.3. I²C LCD Interface 6
3.1.4. LEDs and Buzzer Connections 7
3.1.5. Wiring Summary (Pin Mapping Table) 7
3.2. Software Implementation 7
3.2.1. System and I²C Initialization 8
3.2.2. MPU-6050 Register Configuration 8
3.2.3. Gyroscope Calibration 8
3.2.4. Sensor Data Acquisition 9
3.2.5. Angle Estimation and Complementary Filtering 9
3.2.6. Output and Alarm Control 9
3.2.7. Real-Time Loop Timing 9
4. Conclusions 10
5. References 10

1. Introduction

1.1. Project Description and Requirements

This project focuses on the design and implementation of an Electronic Level using an ATmega328p microcontroller and the MPU-6050 Inertial Measurement Unit (IMU). The primary goal is to calculate and display the orientation (Pitch and Roll) of an object in real-time. Unlike traditional analogic implementations, this digital solution provides precise numerical data, visual feedback through LEDs, and an audible alarm system for safety or specific tilt thresholds.

1.2. Block diagram

The system is organized into three stages: *sensing*, *processing*, and *output*. The MPU-6050 acquires raw accelerometer and gyroscope data and transmits it to the Arduino Uno via I²C. The microcontroller computes the tilt angles using a complementary filter. The results are displayed on a 16×2 I²C LCD, while five LEDs indicate the level zones and an active buzzer provides an acoustic alarm when the tilt exceeds a critical value.

Figure (1) shows a block diagram of the circuit to help with understanding.

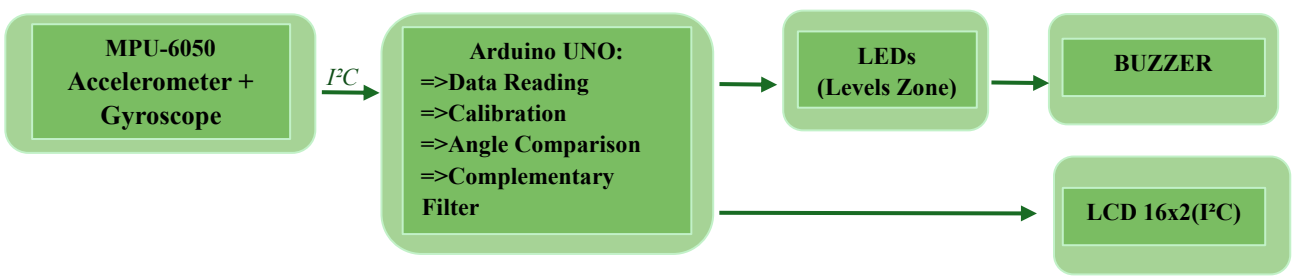

Figure (1). Block Diagram

1.2.1. Project Requirements (Tilt Angle Measurement)

The project requires accurate measurement of the tilt angle along two axes, as shown in Figure (2):

- **Pitch (X)** – rotation around the lateral axis
- **Roll (Y)** – rotation around the longitudinal axis

The system must display the angles in degrees and update them in real time.
Two main constraints are imposed:

- **Accuracy:** accelerometer noise and gyroscope drift must be reduced by combining both sensors using a complementary filter.

- **Response Time:** the processing loop must run at approximately 250 Hz (4 ms) to ensure fast and smooth response.

These requirements define the processing algorithm and the real-time behavior of the embedded system.

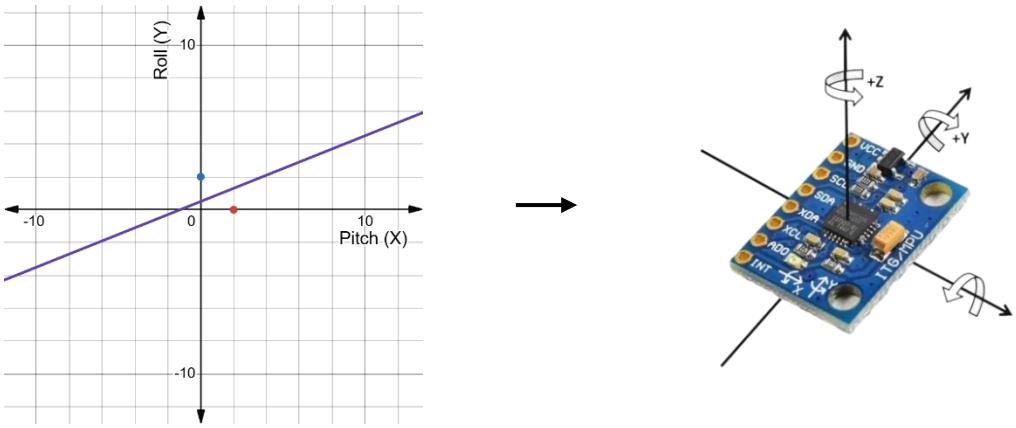

Figure (2): Geometric representation of pitch (X) and roll (Y) angles for the electronic level.

1.2.2. Proposed Solution: MPU-6050 IMU over I²C

The MPU-6050 was chosen as the primary sensor due to its integrated 3-axis accelerometer and 3-axis gyroscope.

- **Communication:** It utilizes the **I²C protocol**, requiring only two wires (SDA and SCL) for communication with the Arduino.
- **Internal Processing:** The sensor features an onboard Digital Motion Processor (DMP), though in this implementation, raw data is accessed directly from the registers (starting at address 0x3B) to allow for custom filtering.
- **Calibration:** To ensure precision, a software calibration routine is performed during startup to calculate the gyroscope's zero-rate offset.

1.2.3. Added Features: I²C LCD Display, LEDs, and Buzzer Alarm

To enhance the user interface and safety of the electronic level, three additional components were integrated:

- **I²C LCD Display**: Provides real-time numerical feedback for Pitch and Roll angles in degrees. The use of the I²C protocol minimizes wiring by sharing the bus with the MPU-6050.
- **LED Array**: Five LEDs are mapped to specific pitch ranges to simulate a visual spirit level. The central LED signals a perfectly leveled state (±1°), while the others indicate the direction of the tilt.

- **Acoustic Alarm (Buzzer)**: An active buzzer triggers a continuous 2kHz warning signal whenever the tilt exceeds a critical threshold of 45° on either axis.

2. Theoretical Background

2.1. MPU-6050 Overview

The MPU-6050 is a compact inertial measurement unit that integrates a 3-axis accelerometer and a 3-axis gyroscope in a single chip. It communicates with the microcontroller via the I²C interface and provides digital motion data.

Its combined sensing capabilities enable accurate and responsive tilt estimation, making it well suited for electronic level applications. See Figure (2) from point 1.2.1.

2.2. Accelerometer Principle (Gravity-Based Tilt)

The accelerometer measures the projection of the gravitational vector on the sensor axes. When the device is tilted, the distribution of the 9.81 m/s² acceleration changes across X, Y, and Z.
The tilt angle can be computed as:

$$\text{Angle}_{acc} = \arctan\left(\frac{A_{axis}}{\sqrt{A_{other1}^2 + A_{other2}^2}}\right) \cdot 57.296 \tag{1}$$

This method provides an absolute reference to gravity but is sensitive to vibrations.

2.3. Gyroscope Principle (Angular Rate)

The gyroscope measures angular velocity in degrees per second. The tilt angle is obtained by integrating this rate over time:

$$\text{Angle}_{gyro} = \text{Angle}_{gyro} + \omega \cdot dt \tag{2}$$

where ω is the measured angular velocity and dt is the sampling period.

This method offers fast response but accumulates error over time, producing drift.

2.4. Basic Angle Estimation and Sensor Fusion (Short Overview)

To compensate for accelerometer noise and gyroscope drift, a complementary filter is used:

$$\text{Angle} = 0.9996 \cdot (\text{Angle} + \text{Gyro_Rate} \cdot dt) + 0.0004 \cdot (\text{Accel_Angle}) \tag{3}$$

The gyroscope provides fast response, while the accelerometer ensures long-term stability, resulting in a smooth and reliable angle estimation.

3. Implementation

3.1. Hardware Implementation

The hardware connections between the Arduino Uno, MPU-6050, I²C LCD, LEDs, and buzzer are illustrated in Figure (3), which presents the full wiring diagram of the electronic level.

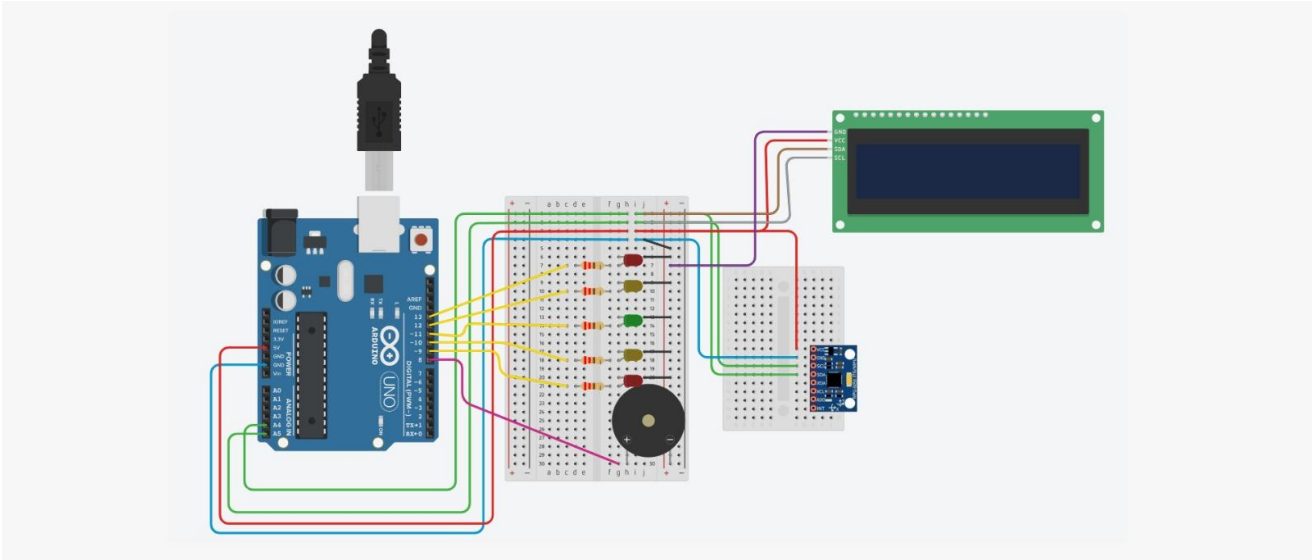

Figure (3): Hardware Implementation Schematic

3.1.1. Arduino Board

The Arduino Uno acts as the central processing unit of the system. It communicates with the MPU-6050 via I²C, computes the tilt angles in real time (250 Hz), and controls the LCD, LEDs, and buzzer. The board also supplies regulated 5 V power to all peripherals, simplifying the hardware design (see Figure (3).

3.1.2. MPU-6050 Connections (Power and I²C)

The MPU-6050 is connected through a four-wire interface for power and data. VCC is linked to the Arduino 5 V output, and GND to the common ground.

Communication uses the I²C bus, with SDA connected to A4 and SCL to A5. These lines are shared with the LCD, both devices operating in parallel on the same bus. In software, the sensor is addressed at 0x68. Short jumper wires are used to ensure stable and noise-free communication.

3.1.3. I²C LCD Interface

A 16×2 character LCD with a PCF8574 I²C adapter is used to display the Pitch and Roll angles in real time. The I²C interface reduces wiring to only four connections (VCC, GND, SDA, SCL) and allows the display to share the same bus (A4, A5) with the MPU-6050.

The module is addressed at 0x27 and is controlled using the NewLiquidCrystal library. The angles are shown with one decimal of precision and a degree symbol (ASCII 223) for clear and professional formatting.

3.1.4. LEDs and Buzzer Connections

Visual and acoustic feedback is provided by five LEDs and an active buzzer, all driven by the Arduino's digital outputs.

The LEDs (pins 9–13) form a visual level indicator, where the central LED lights up when the device is level within ±1°, and the others indicate the direction of tilt.

An active 5 V buzzer connected to pin 8 is triggered when the absolute value of Pitch or Roll exceeds 45°, providing an immediate warning in unsafe orientations.

3.1.5. Wiring Summary (Pin Mapping Table)

To ensure clarity in the hardware implementation, Table (1) summarizes the electrical connections between the Arduino Uno and all peripheral components. All GND connections are tied to a common ground rail.

Component	Arduino Pin	Pin Function	Description
MPU-6050	A4	I²C SDA	Data line for the IMU sensor
MPU-6050	A5	I²C SCL	Clock line for the IMU sensor
I²C LCD	A4	I²C SDA	Data line for the display (shared bus)
I²C LCD	A5	I²C SCL	Clock line for the display (shared bus)
Buzzer	8	Digital Out	Tilt alarm output
LED 1	9	Digital Out	Pitch < −2.01°
LED 2	10	Digital Out	−2.00° to −1.01°
LED 3	11	Digital Out	Level state (±1.00°)
LED 4	12	Digital Out	1.01° to 2.00°
LED 5	13	Digital Out	Pitch > 2.01°

Table (1): Pin mapping between the Arduino Uno and peripheral components.

3.2. Software Implementation

The software architecture is designed as a deterministic real-time system. It configures the MPU-6050 sensor, acquires raw motion data, computes tilt angles, and manages output peripherals within a strict 250 Hz (4 ms) fixed-time loop.

3.2.1. System and I²C Initialization

At startup, the I²C bus is initialized using Wire.begin(), setting the Arduino as master.
The LCD is initialized at address 0x27, enabling the display and backlight for real-time visualization.

```
// Start I2C
Wire.begin();

// LCD init
lcd.begin(16, 2);
lcd.setBacklight(HIGH);
```

3.2.2. MPU-6050 Register Configuration

This function wakes the MPU-6050 from sleep mode and configures the accelerometer to ±8 g and the gyroscope to ±500°/s.
These ranges provide a good compromise between resolution and dynamic range for tilt measurement.

```
void setup_mpu_6050_registers() {
  // MPU-6050 Configuration
  Wire.beginTransmission(0x68);
  Wire.write(0x6B);
  Wire.write(0x00);
  Wire.endTransmission();

  // Accelerometer config (+/-8g)
  Wire.beginTransmission(0x68);
  Wire.write(0x1C);
  Wire.write(0x10);
  Wire.endTransmission();

  // Gyro config (500 dps)
  Wire.beginTransmission(0x68);
  Wire.write(0x1B);
  Wire.write(0x08);
  Wire.endTransmission();
}
```

3.2.3. Gyroscope Calibration

To eliminate zero-rate offsets, the system averages 1000 stationary measurements during startup. These offsets are subtracted from all subsequent readings to minimize long-term integration drift.

```
// Calibrate gyro
  gyro_x_cal = gyro_y_cal = gyro_z_cal = 0;
  for (int cal_int = 0; cal_int < 1000; cal_int++) {
    read_mpu_6050_data();
    gyro_x_cal += gyro_x;
    gyro_y_cal += gyro_y;
    gyro_z_cal += gyro_z;
    delay(3); // ~250Hz
  }
```

```
  gyro_x_cal /= 1000;
  gyro_y_cal /= 1000;
  gyro_z_cal /= 1000;
```

3.2.4. Sensor Data Acquisition

The system requests 14 consecutive bytes starting from register 0x3B. These contain high and low bytes for accelerometer (X, Y, Z), temperature, and gyroscope (X, Y, Z) data, which are reconstructed into 16-bit signed integers using bitwise shifting.

```
Wire.requestFrom(0x68, 14);
  while (Wire.available() < 14);

  acc_x  = (Wire.read() << 8) | Wire.read();
```

3.2.5. Angle Estimation and Complementary Filtering

The software implements a sensor fusion algorithm. Gyroscope data provides immediate updates, while the accelerometer acts as an absolute gravity reference to correct drift.

```
angle_pitch += gyro_x * 0.0000611;
angle_pitch = angle_pitch * 0.9996 + angle_pitch_acc * 0.0004;
```

3.2.6. Output and Alarm Control

The updateLEDs() logic maps the Pitch angle to visual zones, while a safety check uses fabs() to trigger a 2 kHz alarm if tilt exceeds 45° on either axis.

```
// === BUZZER: sound if |pitch| > 45 OR |roll| > 45 ===
  if (fabs(angle_pitch_output) > 45.0 || fabs(angle_roll_output) > 45.0) {
    tone(buzzerPin, 2000);   // 2kHz makes beep
```

This mechanism provides immediate acoustic feedback in unsafe orientations, independent of the sign of the tilt.

3.2.7. Real-Time Loop Timing

To ensure the mathematical validity of the gyroscope integration, the micros() timer enforces a constant 4 ms loop period. This prevents timing jitter from degrading the angle estimation accuracy. By maintaining a deterministic 250 Hz execution rate, the system guarantees stable real-time behavior and correct angle computation.

```
// Keep loop at 250Hz
  while (micros() - loop_timer < 4000);
  loop_timer = micros();
```

4. Conclusions

The *Electronic Level Using MPU-6050 and Arduino* project demonstrates the effective integration of an inertial measurement unit with a microcontroller to solve a practical measurement task. Using the I²C protocol, the system acquires sensor data efficiently and displays real-time results on an LCD with minimal wiring complexity.

The complementary filter plays a key role in achieving stable and accurate angle estimation by combining the fast response of the gyroscope with the long-term stability of the accelerometer. The addition of LED and buzzer feedback transforms the device into a functional safety tool capable of alerting the user when critical tilt thresholds are exceeded.

Overall, the project illustrates a practical application of sensor fusion and real-time embedded processing, and provides a solid foundation for future extensions.

5. References

[1] DroneBot Workshop, "Build an Electronic Level with MPU-6050 and Arduino," 2019. [Online]. Available: https://dronebotworkshop.com/mpu-6050-level/

[2] InvenSense (TDK), "MPU-6050 Product Specification, Revision 3.4." [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf

[3] Arduino, "Wire Library – I²C Communication." [Online]. Available: https://www.arduino.cc/reference/en/language/functions/communication/wire/

[4] F. Malpartida, "NewLiquidCrystal Library (I²C support)." [Online]. Available: https://bitbucket.org/fmalpartida/new-liquidcrystal/

[5] J. Rowberg, "I2Cdevlib: I2C device library for Arduino (including MPU-6050)." GitHub repository. [Online]. Available: https://github.com/jrowberg/i2cdevlib/tree/master

[6] "Complementary filter and relative orientation with MPU6050," Hibit.dev. [Online]. Available: https://www.hibit.dev/posts/92/complementary-filter-and-relative-orientation-with-mpu6050